

The Earliest Deadline Late (EDL) Server

Md Tawhidur Rahman Tuhin
Electronic Engineering
Hochschule Hamm-Lippstadt
Email: md-tawhidur-rahman.tuhin@stud.hshl.de

Abstract—This paper addresses the problem of scheduling both hard *periodic* tasks and unpredictable *aperiodic* requests on a single processor. Periodic tasks must always meet their deadlines, while aperiodic requests should receive the shortest possible response time. The challenge is to use the processor efficiently without affecting the safety of the periodic tasks. We study one solution to this problem, the *Earliest Deadline Late (EDL) server*, a dynamic-priority mechanism built on top of Earliest Deadline First (EDF) scheduling and described by Buttazzo in Section 6.5 of *Hard Real-Time Computing Systems*. The main idea of the EDL server is: schedule the periodic tasks as late as possible, so that the processor becomes free earlier, creating idle time where aperiodic requests can execute without missing any periodic deadline. We explain the theory behind the server, define the server and its algorithm, outline the proofs that the task set stays schedulable as long as $U_p \leq 1$ and that no other on-line method can serve a request faster, discuss the $O(Nn)$ run-time cost, and work through the textbook example in full.

Index Terms—Real-time systems, aperiodic service, dynamic priority, EDF, EDL server, idle time, slack, optimal response time.

I. INTRODUCTION

A *real-time system* is a system whose correctness depends on both the value and the timing of its results [1]. In a *hard* real-time system, missing even a single deadline counts as a failure, and in applications such as flight control or anti-lock braking this failure can be dangerous. The workload of such a system usually has a regular part: *periodic* tasks that are released repeatedly with a fixed period T_i , each needing a worst-case computation time C_i and a relative deadline D_i . These tasks model the recurring sensing and control loops of the system. The system must also handle *aperiodic* work: requests that arrive at unpredictable times, such as an operator command or an asynchronous event. Aperiodic requests are usually *soft*, so missing their (often informal) deadline is not fatal. The goal is to serve them as quickly as possible while the hard periodic deadlines are always satisfied [1, 3].

The two simple service strategies both have limitations. In *background* service, aperiodic requests are executed only when no periodic task is running. This method is safe, because the periodic deadlines are protected, but the response time of aperiodic requests can become very large. In contrast, *immediate* service executes aperiodic requests as soon as they arrive. This provides fast response, but it may delay periodic tasks and cause deadline violations. An *aperiodic server* solves this problem by

reserving a controlled amount of processor capacity for aperiodic requests. It decides when and how much processor time can be used by aperiodic work while ensuring that the hard periodic deadlines are always satisfied. In this way, it provides a balance between safety and response time. When the underlying scheduler is EDF, which is optimal on a single processor and can use it up to full capacity, the result is a *dynamic-priority server*. The EDL server is the member of this family with the best possible aperiodic response time, and it is the subject of this paper.

The rest of the paper is organised as follows. Section II defines the task model and the EDF results we rely on. Section III compares the EDL server with the other dynamic servers of the same chapter, which helps explain why it was developed. Section IV presents EDL scheduling and the availability function. Section V defines the server and its algorithm, and Section VI shows that it is both safe and optimal. Section VII discusses its run-time cost, and Section VIII works through the textbook example step by step.

II. BACKGROUND AND SYSTEM MODEL

We consider n independent, preemptable periodic tasks $\tau_1, \dots, \tau_n$, with $\tau_i = (C_i, T_i)$ and relative deadline $D_i = T_i$. The k -th instance of τ_i is released at $r_{i,k} = (k-1)T_i$ and has absolute deadline $d_{i,k} = r_{i,k} + T_i$. The utilisation of a task and the total *periodic utilisation* are [1, 6]

$$U_i = \frac{C_i}{T_i}, \quad U_p = \sum_{i=1}^n \frac{C_i}{T_i}. \quad (1)$$

U_p tells us how much of the processor is used by all periodic tasks together. The schedule repeats after the *hyperperiod* $H = \text{lcm}(T_1, \dots, T_n)$; this is the time after which the whole task pattern starts again.

A. Earliest Deadline First

Under EDF the ready job with the smallest absolute deadline is executed. EDF is optimal for preemptive uniprocessor scheduling, and a periodic set with $D_i = T_i$ is schedulable *if and only if* [6]

$$U_p = \sum_{i=1}^n \frac{C_i}{T_i} \leq 1. \quad (2)$$

This is the EDF schedulability test. If $U_p \leq 1$, all periodic tasks can meet their deadlines. If $U_p > 1$, the processor is overloaded and some deadlines may be missed.

B. Response Time and Slack

For an aperiodic request J_a released at r_a , requiring C_a units and finishing at f_a , the *response time* is [1]

$$R_a = f_a - r_a \geq C_a, \quad (3)$$

with equality only when J_a runs with no waiting. The *slack* (or *laxity*) of an active job with remaining computation $c_i(t)$ and deadline d_i is $L_i(t) = d_i - t - c_i(t)$. Slack means safe waiting time: it tells us how long we can delay a periodic task without missing its deadline. The EDL server is, in short, a method that turns this periodic slack into early service for aperiodic requests.

III. CONTEXT: DYNAMIC-PRIORITY SERVERS

Before defining the EDL server, it is useful to compare it with the simpler dynamic servers of the same chapter, because it was developed as an improvement of them. The *Dynamic Priority Exchange (DPE)* server (the EDF counterpart of the fixed-priority Priority Exchange server [8]) saves unused processor time by trading it with periodic tasks that have earlier deadlines, and gives this saved time to aperiodic requests later. The *Dynamic Sporadic Server (DSS)* (the EDF version of the Sporadic Server [7]) has a fixed capacity (C_s, T_s) ; after using its capacity, it waits one period before getting that capacity back. The *Total Bandwidth Server (TBS)* is even simpler: every new request is assigned the deadline [2, 3]

$$d_k = \max(r_k, d_{k-1}) + \frac{C_k}{U_s} \quad (4)$$

and EDF then schedules the request according to that deadline. All three servers are feasible exactly when $U_p + U_s \leq 1$ [3].

However, TBS reserves only a fixed amount of processor bandwidth. Sometimes the running periodic tasks still have extra unused slack, but TBS cannot use it, so some requests wait longer than necessary [1]. The EDL server uses *all* the available slack of the periodic tasks. Because of this, aperiodic requests start earlier and finish faster. It is the dynamic-priority form of the older slack-stealing idea [5].

IV. EDL SCHEDULING AND THE AVAILABILITY FUNCTION

The EDL server is built on two complementary versions of EDF introduced by Chetto and Chetto [4]. In the first, *EDS* (Earliest Deadline as Soon as possible), jobs run as early as possible; this is ordinary EDF. In the second, *EDL* (Earliest Deadline as Late as possible), every job is delayed to the latest time that still meets its deadline. The difference is where the idle time appears. Under EDS the work is done as soon as possible, so the idle time accumulates late. Under EDL the work is done as late as safely possible, so the idle time appears early. EDL creates idle time as early as possible without causing any periodic task to miss its deadline, and it is exactly this early free time that the server needs.

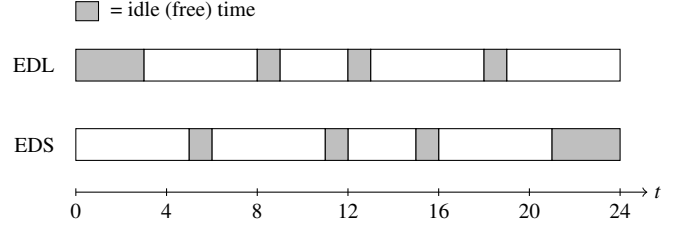


Fig. 1. EDS versus EDL for $\tau_1=(3,6)$, $\tau_2=(2,8)$. Both schedules meet all deadlines and leave the same total idle time (6 units), but EDS leaves it *late* while EDL exposes it *early* (gray). The EDL server exploits the early idle time of the lower schedule. Task set and EDL idle pattern from [1, Sec. 6.5]; EDS/EDL duality from [4].

A. Availability Function

For a scheduling algorithm A and a task set J , the *availability function* records when the processor is free [1, Sec. 6.5], [4]:

$$\omega_J^A(t) = \begin{cases} 1 & \text{if the processor is idle at } t, \\ 0 & \text{otherwise,} \end{cases} \quad (5)$$

and its integral $\Omega_J^A(t_1, t_2) = \int_{t_1}^{t_2} \omega_J^A(t) dt$ is the total idle time in $[t_1, t_2]$. The property that makes EDL useful is the following.

Theorem 1 (Maximum idle time [4, Th. 2], [1, Th. 6.4]). *For any preemptive algorithm A and any instant t ,*

$$\Omega_J^{\text{EDL}}(0, t) \geq \Omega_J^A(0, t). \quad (6)$$

This means that in every interval $[0, t]$, EDL provides the maximum amount of idle time that any scheduling algorithm can achieve.

B. Representation by Two Arrays

Because the EDL idle pattern repeats every hyperperiod H , it can be stored very compactly as two lists [1, Sec. 6.5], [4],

$$\mathcal{E} = (e_0, e_1, \dots, e_p), \quad \mathcal{D} = (\Delta_0, \Delta_1, \dots, \Delta_p), \quad (7)$$

one giving the start e_i of each idle interval and the other its length Δ_i . Idle intervals only ever begin just after a periodic instance is released. Together these two arrays are the server's map of free time; Fig. 2 shows them for the example of Section VIII.

C. EDS versus EDL on the Same Task Set

Theorem 1 is easiest to see by comparing the two schedules of the same task set. Fig. 1 does this for $\tau_1=(3,6)$, $\tau_2=(2,8)$. Both schedules meet every deadline, and over a full hyperperiod both leave the same total of 6 idle units. The difference is the position of this idle time. EDS finishes the periodic work first, so its free time appears late; EDL delays the periodic work, so its free time appears early. An aperiodic request can only be served from the moment it arrives onward. Early idle time is therefore useful, because it allows an aperiodic request to begin execution immediately after it arrives. This is why the server is built on the EDL schedule and not on the EDS one.

Algorithm 1 EDL aperiodic server

Inputs: periodic set τ (parameters known); aperiodic queue Q .

```
1: while system runs do
2:   if  $Q = \emptyset$  then
3:     schedule  $\tau$  by EDF (EDS)
4:   else if a new request  $J$  just arrived (no active aperiodic)
5:     build EDL schedule of current/future instances of  $\tau$ 
6:     extract idle map  $(\mathcal{E}, \mathcal{D})$  from  $\omega^{\text{EDL}}$ , Eq. (7)
7:   end if
8:   if  $Q \neq \emptyset$  and current instant is idle in  $(\mathcal{E}, \mathcal{D})$  then
9:     run head of  $Q$  for one unit; decrement its  $C_a$ 
10:    if  $C_a = 0$  then set  $f_a$ ; dequeue  $J$ 
11:  else
12:    run highest-priority (earliest-deadline) periodic job
13:  end if
14: end while
```

V. THE EARLIEST DEADLINE LATE SERVER

A. Definition and Operating Principle

The EDL server, proposed by Spuri and Buttazzo [2, 3], uses the idle time of an EDL schedule to run aperiodic requests as early as possible. The server rule is as follows:

- If no aperiodic request is waiting, the processor runs the periodic tasks using ordinary EDF (that is, EDS).
- When an aperiodic request arrives, and no earlier request is still being served, the server computes the early idle time available in the EDL schedule of the *current* periodic instances (with releases and deadlines measured from the present instant). The request is executed in those idle slots, starting with the earliest ones.
- If the current time is not free, the processor runs the periodic task with the earliest deadline. This keeps all periodic deadlines safe.

The aperiodic request does not need a separate deadline, because it only runs during idle time that already exists in the EDL schedule; the server simply places the request into those safe idle intervals. A useful side effect is that the server automatically devotes exactly the leftover bandwidth [1, Sec. 6.5], [3]

$$U_s = 1 - U_p \quad (8)$$

to aperiodic work. Here U_p is the processor utilisation of all periodic tasks, and U_s is the remaining processor bandwidth available for aperiodic requests. There is no parameter to tune, and the response times are optimal (Section VI). One practical simplification is worth noting: at each arrival the server does not need to rebuild the whole idle map, but only the part that lies before the longest deadline among the periodic instances currently active [1].

B. Algorithm

Algorithm 1 states the procedure. The idle-time computation of lines 5–6 is the EDL routine of [4]; equivalently, one unit

of idle time exists at t exactly when the system slack $S(t) = \min_{d>t} [(d-t) - g(t, d)]$ is positive, where $g(t, d)$ is the periodic processor demand with deadline $\leq d$.

How it runs. When the queue is empty, the periodic tasks simply run under EDF (line 3). The first time a request arrives, the server computes the EDL idle map for the current state of the periodic tasks (lines 5–6). From then on, whenever the map marks the current instant as free, the waiting request runs for one unit (line 9), so it advances as fast as the slack allows. Whenever the map is not free, the earliest-deadline periodic job runs instead (line 12), and this is what keeps the hard deadlines safe. The request completes (line 10) at the end of the last free slot it uses.

VI. PROPERTIES

A. Schedulability

Theorem 2 (Schedulability [3], [1, Th. 6.5]). *Given n periodic tasks with utilisation U_p and the corresponding EDL server, the whole set is schedulable if and only if $U_p \leq 1$.*

Proof sketch. (If) The key point is that the server does not reduce the processor time available to the periodic tasks; it only uses processor time that would otherwise remain idle. The condition $U_p \leq 1$ is sufficient for EDF, hence also for EDL, which is a particular EDF implementation. Since the periodic tasks are still scheduled by EDF, and aperiodic requests execute only during the precomputed idle times, the timeliness of the periodic tasks is unaffected and no deadline is missed. (Only if) If the set is schedulable with the server, it is also schedulable without it, so $U_p \leq 1$ by (2). $\square$

B. Optimality

Lemma 1 (Optimality [3], [1, Lem. 6.3]). *Let A be any on-line preemptive algorithm and J_i an aperiodic request. If $f^A(J_i)$ is its finishing time under A , then*

$$f_{\tau \cup \{J_i\}}^{\text{EDL server}}(J_i) \leq f_{\tau \cup \{J_i\}}^A(J_i). \quad (9)$$

Proof sketch. The optimality proof is based on Theorem 1. Suppose J_i arrives at t and let $\tau(t)$ be the active and future periodic instances. Any algorithm A executes J_i during idle periods of the periodic schedule it produces. Moving the time origin to t and applying Theorem 1 to $\tau(t)$ gives, for every $t' \geq t$, $\Omega_{\tau(t)}^{\text{EDL}}(t, t') \geq \Omega_{\tau(t)}^A(t, t')$: EDL always provides the maximum amount of early idle time. The EDL server places J_i exactly into these earliest idle intervals, so it collects the required C_i units of service no later than A does; hence $f_{\tau \cup \{J_i\}}^{\text{EDL server}}(J_i) \leq f_{\tau \cup \{J_i\}}^A(J_i)$. $\square$

As a result, no other on-line scheduling algorithm can complete the request earlier while still preserving all periodic deadlines. The EDL response times are the best achievable, which is why they are often used as a reference for evaluating simpler, more practical servers.

VII. RUN-TIME COST

The main problem of the EDL server is the computational cost of calculating the idle times. Building the idle-time map re-

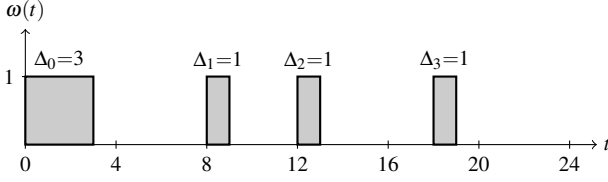


Fig. 2. Availability function $\omega^{\text{EDL}}(t)$ of the example set $\tau_1=(3,6)$, $\tau_2=(2,8)$. EDL pushes idle time to the front: $\mathcal{E}=(0,8,12,18)$, $\mathcal{D}=(3,1,1,1)$, total 6 = $H(1-U_p)$ units per hyperperiod. Adapted from [1, Fig. 6.7 and Table 6.1].

quires, in the worst case, [1, Sec. 6.5], [3]

$$O(Nn) \quad (10)$$

operations, where n is the number of periodic tasks and $N = \sum_{i=1}^n H/T_i$ is the total number of periodic task instances in one hyperperiod. If the task periods are nearly coprime, the hyperperiod becomes very large, which increases N significantly. Therefore, recalculating the idle-time map whenever a new aperiodic request arrives becomes too expensive for practical on-line systems. The cost can be reduced by recomputing only the idle times before the longest deadline of the active instances, but it stays high. For this reason the exact EDL server is mostly used as a theoretical best case, and as the starting point for the *Improved Priority Exchange* server [3].

A. From EDL to a Practical Server

The Improved Priority Exchange (IPE) server solves this problem by moving the computation from on-line execution to off-line preparation. The idle pattern $(\mathcal{E}, \mathcal{D})$ is calculated only once, before system operation. During runtime, the server only updates the available aperiodic capacity by Δ_i units at the predefined instants $t = e_i + kH$, mirroring the function ω^{EDL} exactly. Instead of recalculating the whole schedule, unused capacity is exchanged with the execution of periodic tasks and reclaimed later. Because the refills follow the EDL idle times, the server always runs at the highest priority, and its response times match those of the exact EDL server in all but a few rare cases. This reduces the runtime overhead while maintaining the same utilisation guarantee $U_p \leq 1$ [3].

VIII. WORKED EXAMPLE

To make this concrete, we follow the example from the textbook. There are two periodic tasks,

$$\tau_1 = (C_1=3, T_1=6), \quad \tau_2 = (C_2=2, T_2=8),$$

with $D_i = T_i$. From (1), $U_p = \frac{3}{6} + \frac{2}{8} = 0.75 \leq 1$, so the set is feasible by (2); the leftover bandwidth is $U_s = 1 - U_p = 0.25$ and the hyperperiod is $H = \text{lcm}(6,8) = 24$. A single aperiodic request J_a arrives at $r_a = 8$ and needs $C_a = 4$ units.

Step 1: the EDL idle map. Running τ_1 and τ_2 as late as possible across $[0, 24]$ gives the availability function in Fig. 2, whose idle intervals are [1, Table 6.1]

$$\mathcal{E} = (0, 8, 12, 18), \quad \mathcal{D} = (3, 1, 1, 1).$$

The idle time over one hyperperiod adds up to $\Omega^{\text{EDL}}(0, 24) = 3 + 1 + 1 + 1 = 6 = H(1 - U_p)$, which matches (8): the server has six free units to give away each hyperperiod.

Step 2: serving the request. The slack tells us how long the processor can postpone the periodic tasks. At $t = 8$ it is, computed as in [4] and [1, Sec. 6.5],

$$S(8) = \min\{(12-8) - 1, (16-8) - 3, (18-8) - 6, (24-8) - 11\} = 3, \quad (11)$$

where each term is a future deadline minus the periodic work due by it. The smallest slack is 3, so the request can run for 3 units before any periodic task is forced to execute. The request uses the processor from $t = 8$ to $t = 11$. At $t = 11$, τ_1 has to run because it cannot be delayed anymore: its deadline is at 12, and waiting longer would make it miss the deadline. After τ_1 runs, the request gets the processor again and its fourth and final unit runs at $t = 12$. It therefore finishes at

$$f_a = 13, \quad R_a = f_a - r_a = 5.$$

Fig. 3 shows the full schedule, and every periodic instance still meets its deadline—for example, τ_2 's second instance (released at 8, deadline 16) completes at $t = 15$.

Step 3: is this really the best possible? With plain *background* service the request would have to wait for the idle slots of the EDS schedule and would finish much later. No on-line policy can finish it before $t = 13$, so the EDL response time $R_a = 5$ is the best achievable, exactly as Lemma 1 states.

IX. DISCUSSION

The example shows the two properties that make the EDL server special. It is *optimal*: the request is placed into the earliest free time the periodic set can possibly give it, so no method could finish it sooner. And it is *safe*: the request only runs in time the periodic tasks would have left idle anyway, so no hard deadline is ever at risk and the feasibility test remains the plain EDF test $U_p \leq 1$. In addition, the server manages its own bandwidth—it uses exactly the spare fraction $1 - U_p$ of the processor, with no manual parameter.

The main drawback is the computation cost. The EDL server is mainly used when the periodic task set is fixed and the goal is to achieve the best possible response time for aperiodic requests. Since EDL is optimal, it is useful as a reference method for analysis and comparison with other servers. However, its high computation cost makes it less suitable for systems where tasks or parameters change frequently. The IPE server is preferred in practical real-time systems where on-line efficiency is important. It calculates the idle pattern off line and avoids expensive recomputation during runtime. Therefore, IPE provides almost the same response performance as EDL but with much lower runtime overhead, and it is a better choice for systems that require predictable execution with changing aperiodic requests.

X. CONCLUSION

This paper presented the Earliest Deadline Late server as described in Section 6.5 of Buttazzo's *Hard Real-Time Computing*

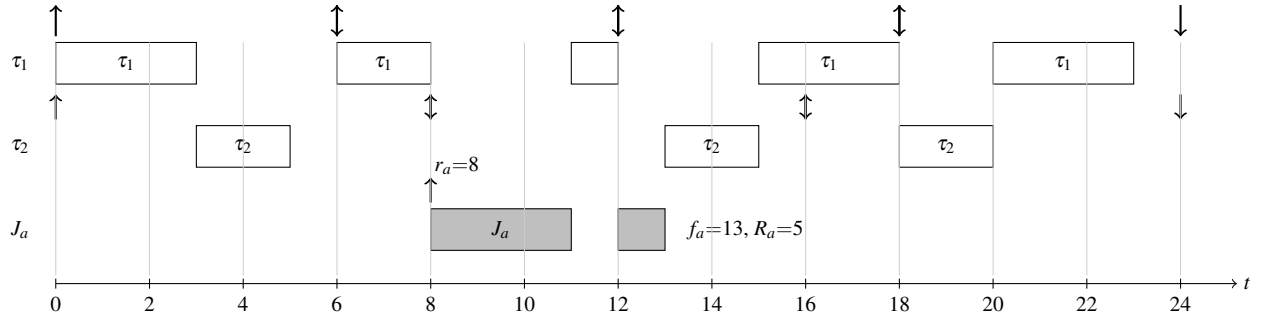


Fig. 3. EDL-server schedule for the textbook example ($\tau_1=(3,6)$, $\tau_2=(2,8)$). Up arrows mark releases, down arrows mark deadlines. The request J_a ($C_a=4$) arriving at $t=8$ is served in the early idle time exposed by EDL (units at $t=8, 9, 10, 12$) and finishes at $f_a=13$; all periodic deadlines are met. This response time is optimal (Lemma 1). Adapted from [1, Fig. 6.8].

Systems. The server is the dynamic-priority, EDF-based version of slack stealing: by delaying the periodic tasks as far as their deadlines allow, it frees the processor as early as possible (Theorem 1) and then runs aperiodic requests in exactly that free time. This gives two guarantees. First, every periodic deadline is kept as long as $U_p \leq 1$ (Theorem 2), because the requests only use processor time that would otherwise remain idle. Second, aperiodic requests are served as fast as any on-line method can serve them (Lemma 1), and the whole leftover bandwidth $1 - U_p$ is assigned to aperiodic work automatically. Because of its $O(Nn)$ recomputation cost, the exact server is most valuable as a theoretical optimum and a baseline for comparison, with the Improved Priority Exchange server serving as the practical, near-optimal alternative. The worked example illustrated both results: a four-unit request was placed in the EDL idle time and finished at the earliest time any on-line algorithm could achieve.

REFERENCES

- [1] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. New York, NY, USA: Springer, 2011, ch. 6, sec. 6.5.
- [2] M. Spuri and G. C. Buttazzo, "Efficient aperiodic service under earliest deadline scheduling," in *Proc. IEEE Real-Time Systems Symp. (RTSS)*, San Juan, Puerto Rico, 1994, pp. 2–11.
- [3] M. Spuri and G. C. Buttazzo, "Scheduling aperiodic tasks in dynamic priority systems," *Real-Time Systems*, vol. 10, no. 2, pp. 179–210, 1996.
- [4] H. Chetto and M. Chetto, "Some results of the earliest deadline scheduling algorithm," *IEEE Trans. Softw. Eng.*, vol. 15, no. 10, pp. 1261–1269, Oct. 1989.
- [5] J. P. Lehoczky and S. Ramos-Thuel, "An optimal algorithm for scheduling soft-aperiodic tasks in fixed-priority preemptive systems," in *Proc. IEEE Real-Time Systems Symp. (RTSS)*, Phoenix, AZ, USA, 1992, pp. 110–123.
- [6] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *J. ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973.
- [7] B. Sprunt, L. Sha, and J. P. Lehoczky, "Aperiodic task scheduling for hard real-time systems," *Real-Time Systems*, vol. 1, no. 1, pp. 27–60, 1989.
- [8] J. P. Lehoczky, L. Sha, and J. K. Strosnider, "Enhanced aperiodic responsiveness in hard real-time environments," in *Proc. IEEE Real-Time Systems Symp. (RTSS)*, San Jose, CA, USA, 1987, pp. 261–270.